

Unit Testing with JUnit 5 in Java

CSE 114: Introduction to Object-Oriented Programming

Ahmad Esmaili

Stony Brook University

<http://www.cs.stonybrook.edu/~cse114>

Testing vs. Debugging

- **Testing:** Execute a program with selected inputs to reveal defects.
 - Design of test cases to maximize the chances of detecting defects.
 - Testing can show the presence of bugs, but **cannot guarantee** their absence.
- **Debugging:** Identifying, isolating, and fixing the cause of incorrect behavior.
 - Runtime errors (e.g., null pointer)
 - Logical errors (e.g., wrong calculation)

Important Definitions

- **Testing**

- A process of executing a program with a set of test cases and comparing the actual results with the expected results to detect defects.

- **Verification**

- The process of determining whether the software correctly implements the specified design or requirements.

Often phrased as: *“Did we build the system correctly?”*

- **Validation**

- The process of determining whether the software meets the user’s needs and requirements.

Often phrased as: *“Did we build the right system?”*

- performed through testing and verification techniques.

- **Defensive Programming**

- A programming practice in which code is written to anticipate and handle potential errors or misuse, making programs easier to debug, validate, and maintain.

Kinds of Testing

- Unit Testing
 - Test each module in a program separately, e.g., `Calculator.add()`.
- Integration Testing
 - Test interfaces between modules, e.g., `Calculator + Logger` (harder due to interactions).
 - Often more challenging than unit testing because of module interactions
- Regression Testing
 - Test programs after modifications to ensure correct behavior of the original program is preserved, e.g., retest **old features** after new patch.
- System Testing
 - Test overall system behavior, e.g., end-to-end user workflow.

Aspects of Testing

- How do we generate test cases?
 - **Exhaustive**
 - Consider all possible combinations of inputs.
 - Often infeasible (huge or infinite inputs)
 - Example: 2 integers from 1 to 1,000 $\rightarrow$ 1,000,000 combinations
 - **Sampled**
 - Use a representative subset of possible inputs.
 - Cover typical, edge, and invalid inputs cases.
 - **Strategies**
 - Black-box: Based on specifications, not code
 - Glass-box: Based on internal code structure

Black-box testing

- The best place to start when attempting to test a program thoroughly
- Based on specification, not implementation
 - Advantage: resilient to code refactoring, focuses on requirements.
- Test cases are not affected by:
 - Invalid assumptions made by the programmer
 - Implementation changes
 - Same test cases can be reused after program structures has changed
- Test cases can be generated by an “independent” tester, unfamiliar with the implementation.
- Test cases should cover all important behaviors described in the specification, including exceptional cases.

Glass-box (white-box) testing

- Black-box testing is generally not enough
- Test cases are designed using knowledge of the program's internal structure.
- ***Path-completeness:***
 - Test cases are generated to exercise each path *through* a program, e.g., statement coverage, branch coverage.
 - May still miss some errors.
 - Impractical for large programs (path explosion).
 - Best for small program fragments with a reasonable number of paths.

Boundary Conditions

- A *boundary condition* is an input value at the edge (or limit) of a valid input range.
- Helps catch two common types of errors:
 - **Logical errors:** Occur when a program fails to correctly handle special cases at the boundaries of input ranges (for example, missing a condition for the smallest or largest possible input).
 - **Runtime or system errors:** Occur when a program fails to check conditions that may cause the programming language or hardware to raise an exception, such as arithmetic overflow, invalid indexing, division by zero.
- Example: Test cases where array size is 0 (empty), 1 (minimum valid), MAX (maximum allowed)

Testing paths through specification

- Examine the method specifications (preconditions) & all paths through method to generate unique test cases. Example:

```
/* REQUIRES: x >= 0 && y >= 10 */
```

```
public static int calc(int x, int y)
```

...

- Translate paths to test cases (including inputs that violate preconditions):

x = 0, y = 10	(x == 0 && y == 10)	✓
x = 5, y = 10	(x > 0 && y == 10)	✓
x = 0, y = 15	(x == 0 && y > 10)	✓
x = 5, y = 15	(x > 0 && y > 10)	✓
x = -1, y = 10	(x < 0 && y == 10)	✗
x = -1, y = 15	(x < 0 && y > 10)	✗
x = -1, y = 9	(x < 0 && y < 10)	✗
x = 0, y = 9	(x == 0 && y < 10)	✗
x = 1, y = 9	(x > 0 && y < 10)	✗

JUnit

- JUnit – Unit Testing Framework for Java
 - Open-source software
 - Hosted on GitHub
 - Not in the standard JDK – Must be added as a dependency (e.g., Maven/Gradle)
- Imports for JUnit 5
 - `import org.junit.jupiter.api.Test;`
 - `import static org.junit.jupiter.api.Assertions.*;`
- `@Test` annotation marks test methods

Junit (Cont'd)

- Common assertions:
 - assertEquals(expected, actual)
 - assertTrue(condition)
 - assertFalse(condition)
 - assertEquals(obj1, obj2)
 - assertThrows(Exception.class, () -> method())
- Test Classes
 - Associate one test class per unit/class
 - Each test class contains one or more test methods
 - Test methods are void and annotated with `@Test` (no need to be public)

Building unit tests with JUnit

- Initialize instance variables or test data (e.g., using `@BeforeEach`)
- Define tests for important cases:
 - null / empty inputs
 - correctness of output
 - boundary / edge cases
- Use assertions to verify behavior:
 - `assertEquals`, `assertTrue`, `assertFalse`, `assertThrows`, ...
- Each test method should test one logical behavior.
- Optionally group related tests (e.g., test classes or suites)

JUnit Example – StatCompiler.java

```
public class StatCompiler {  
    // a, b, & c must all be non-negative  
    public static int averageOfPosInts(int a, int b, int c)  
        throws IllegalArgumentException {  
  
        if ((a < 0) || (b < 0) || (c < 0))  
            throw new IllegalArgumentException("No neg values");  
        int sum = a + b + c;  
        return sum/3;  
    }  
  
    // NOTE: median method contains an intentional error for testing purposes  
    public static int median(int a, int b, int c) {  
        if ( (a >=b) && (a <=c))          return a;  
        else if ((a >= b) && (a >=c))      return b;  
        else                               return c;  
    }  
}
```

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class StatCompilerTest {
    @Test
    void testAverageOfPosInts() {
        System.out.println("testAverageOfPosInts");
        assertEquals(2, StatCompiler.averageOfPosInts(1, 2, 3));

        assertThrows(IllegalArgumentException.class, () -> {
            StatCompiler.averageOfPosInts(-1, 2, 3);
        });
    }

    @Test
    void testMedian() {
        System.out.println("testMedian");
        assertEquals(2, StatCompiler.median(1, 2, 3));
        // may fail due to bug
        assertEquals(2, StatCompiler.median(3, 2, 1));
    }
}
```

StatCompilerTest.java (continued)

```
[INFO] Running StatCompilerTest
[testAverageOfPosInts] PASSED
[testMedian] FAILED
```

Failures:

```
1) testMedian(StatCompilerTest)
   expected: <2> but was: <3>
   at StatCompilerTest.testMedian(StatCompilerTest.java:42)
   Reason: Intentional bug in median() method
```

Summary:

```
Tests run: 2, Failures: 1, Errors: 0, Skipped: 0
Result: FAILED
Elapsed time: 0.01 s
```

Execution Results of StatCompilerTest

♥ Thanks for using JUnit! Support its development at <https://junit.org/sponsoring>

```
testAverageOfPosInts
testMedian
```

```
├─ JUnit Platform Suite ✓
├─ JUnit Jupiter ✓
│   └─ StatCompilerTest ✓
│       ├── testAverageOfPosInts() ✓
│       └─ testMedian() ✗ expected: <2> but was: <3>
└─ JUnit Vintage ✓
```

Failures (1):

JUnit Jupiter:StatCompilerTest:testMedian()

MethodSource [className = 'StatCompilerTest', methodName = 'testMedian', methodParameterTypes = '']

```
=> org.opentest4j.AssertionFailedError: expected: <2> but was: <3>
    org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
    org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow(AssertionFailureBuilder.java:132)
    org.junit.jupiter.api.AssertEquals.failNotEqual(AssertEquals.java:197)
    org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:150)
    org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:145)
    org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:531)
    StatCompilerTest.testMedian(StatCompilerTest.java:24)
    java.base/java.lang.reflect.Method.invoke(Method.java:565)
    java.base/java.util.ArrayList.forEach(ArrayList.java:1604)
    java.base/java.util.ArrayList.forEach(ArrayList.java:1604)
```

Test run finished after 31 ms

```
[ 4 containers found ]
[ 0 containers skipped ]
[ 4 containers started ]
[ 0 containers aborted ]
[ 4 containers successful ]
[ 0 containers failed ]
[ 2 tests found ]
[ 0 tests skipped ]
[ 2 tests started ]
[ 0 tests aborted ]
[ 1 tests successful ]
[ 1 tests failed ]
```

WARNING: Delegated to the 'execute' command.
This behaviour has been deprecated and will be removed in a future release.
Please use the 'execute' command directly.

Java Code Conventions

- Why have code conventions?
 - ~80% of a software's lifetime cost is spent on maintenance and improvement
 - Hardly any software is maintained for its whole life by the original author.
 - Conventions improve readability, maintainability, and ease of enhancement → engineers understand new code faster.
 - **Well-structured code is easier to test and verify using unit tests (e.g., JUnit)**
 - If you ship your source code, it should be as clean and well-packaged as any product
 - Would you want to buy poorly written code?